

1. (2.5pts) Considere um grafo não-dirigido e não-ponderado $G = (V, E)$ e um vértice $s \in V$. Considere também que G foi submetido a uma busca em largura partindo do vértice s e como resultado, obteve-se como retorno as estruturas de dados D e A . De posse do grafo G , do vértice s , das estruturas D e A é possível afirmar que o procedimento abaixo encontra uma árvore de busca em largura com vértice de origem em v , sabendo que v é vizinho de s em G ? Justifique (se ajudar, utilize um contraexemplo).

```

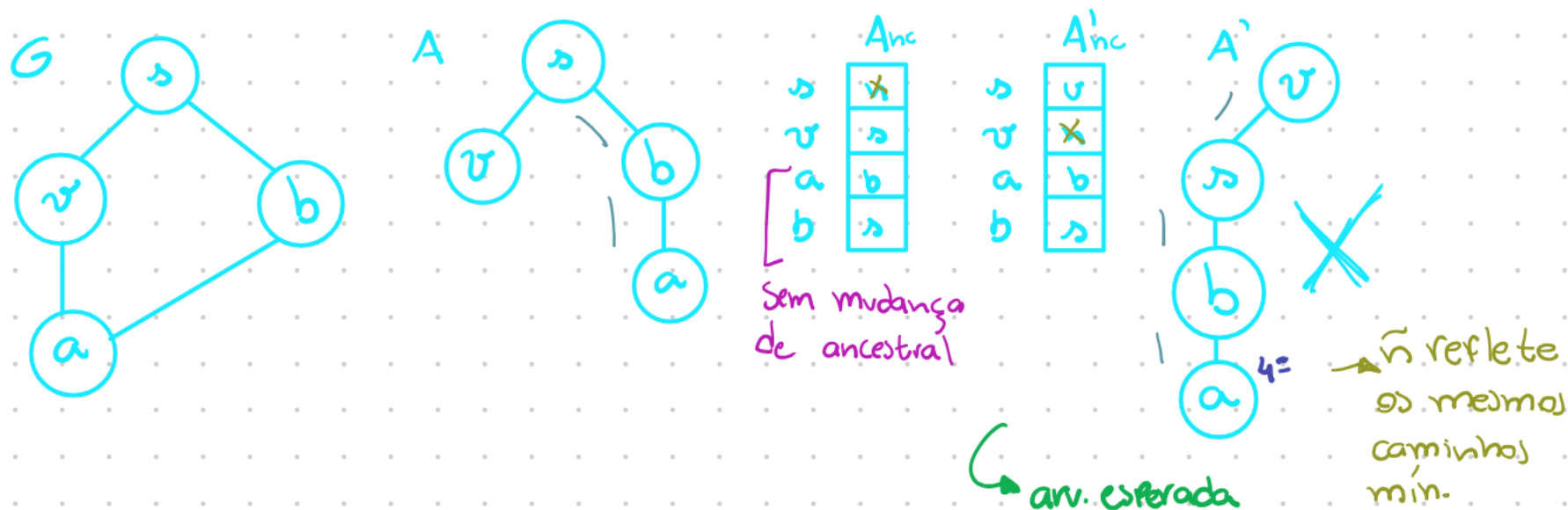
Input  : um grafo  $G = (V, E, w)$ ,  $s, v, D, A$ 
// criar  $A'$  e copiar tudo de  $A$  para  $A'$ 
1  $A' \leftarrow A$ 
// configura-se a nova árvore de busca em largura de  $A'$  com raiz em  $v$ 
2  $A'_v \leftarrow \text{null} \rightarrow v$  sem ancestral
3  $A'_s \leftarrow v \rightarrow$  ancestral de  $s$  é  $v$ 
4 return  $A'$ 

```

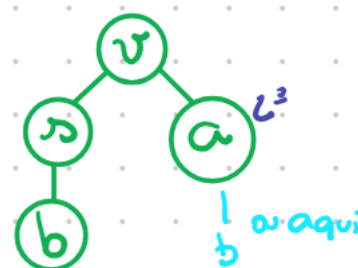
1) $G = (V, E)$
 $s \in V$
 $BL \rightarrow D, A$ } temos

uma nova arv. BL com raiz em v } queremos
 $(s \in N(v))$

Não, a maneira mais fácil de provar é achar um contra-exemplo



R: Não, no contra exemplo é possível verificar que no grafo G com a árvore A , o algoritmo falharia em criar a nova arv. BL, pois sua resposta A' coloca o vértice "a" no 4º nível no lugar do 2º



2. (2.5pts) Dado um grafo $G = (V, E)$, considere que H seja um conjunto de todos os caminhos (não ciclos) hamiltonianos de G . Crie um algoritmo que receba G e H e determine se existe um ciclo hamiltoniano em G .

2) $G = (V, E)$

$H = \{p: p \text{ é um cam. Hamiltoniano que não é ciclo}\}$

Ciclo hm é um Caminho hm onde o último v e o primeiro estão conectados por uma aresta

Alg 2.2

Entrada: $G = (V, E)$, H

1. For each $p \in H$ do
2. if $\{p_i, p_v\} \in E$ then $\rightarrow \text{ou } p_i \in N(p_v)$
3. return true
4. return false

↳ Pseudo-código pode ser válido

R: O algoritmo está acima

3. (2.5pts) Depois de rodar o algoritmo de Floyd-Warshall sobre um grafo dirigido e ponderado $G = (V, A, w)$, é possível determinar se será necessário recalcular os caminhos mínimos apenas com os resultados do algoritmo (matrizes D e Π), considerando que um dos vértices de G foi removido (removido depois de executar o algoritmo)? Justifique.

3) $G = (V, A, w)$

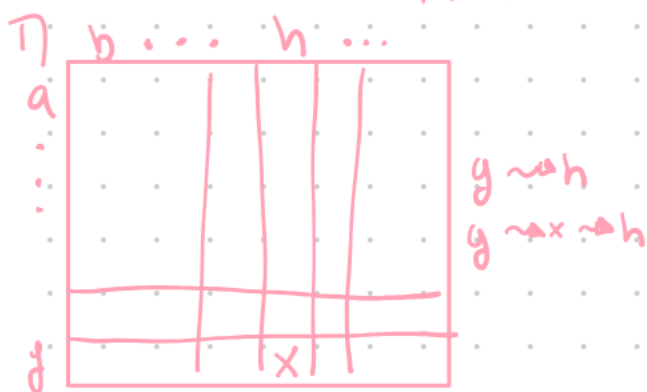
FW $\rightarrow D, \Pi$

depois $x \in V$ é removido

Possso determinar?

é necessário recalcular?

Se um cam. tiver x , será recalculado, p/ verificar olhamos p/ matriz Π



R: Sim. Verificando a matriz de predecessores Π , será descoberto se x é ancestral direto de algum caminho.

* Se x não está presente em Π , ele não é utilizado

p/ os caminhos mínimos encontrados pela execução de FW. Desse modo, não seria necessário recalcular os caminhos mínimos.

* Considere x o vértice removido

4. (2.5pts) Considere o seguinte problema: dado um grafo dirigido e ponderado $G = (V, A, w : A \rightarrow \mathbb{R})$ e os vértices $s, x, y, t \in V$, encontre o caminho $p = \langle s, \dots, x, \dots, y, \dots, t \rangle$ com o menor custo possível. Considerando os algoritmos de caminho mínimo visitados em aula na disciplina, crie um algoritmo para resolver o problema proposto em menor tempo computacional possível.

4) $G = (V, A, w : A \rightarrow \mathbb{R})$

s, x, y, t

encontrar o cam. mínimo $p = \langle s, \dots, x, \dots, y, \dots, t \rangle$?

criar algoritmo p/ encontrar p no menor tempo possível

$n = |V|$ FW = $O(n^3)$

$m = |E|$ BF = $O(nm)$

Q: O algoritmo está abaixo, quebrando em 3 partes

Alg 24

Entrada: $G = (V, A, w), s, x, y, t \in V$

1. $(r, D, A) \leftarrow \text{BF}(G, s)$
2. if $r = \text{false}$ or $Dx = \infty$ then return null
3. $p \leftarrow \text{montaCaminho}(A, x, x)$
4. $(r', D', A') \leftarrow \text{BF}(G, x)$
5. if $r' = \text{false}$ or $D'y = \infty$ then return null
6. $p' \leftarrow \text{montaCaminho}(A', x, y)$
7. $(r'', D'', A'') \leftarrow \text{BF}(G, y)$
8. if $r'' = \text{false}$ or $D''t = \infty$ return null
9. $p'' \leftarrow \text{montaCaminho}(A'', y, t)$
10. return $p \cdot p' \cdot p''$

$s \rightarrow x$,
Se inalcançável or ciclo-, Para
trecho extraído

$x \rightarrow y$

$y \rightarrow t$

→ concatena os caminhos

* em FW não teria como omitir origem

Alg montaCaminho

Entrada: A, v, d

1. $p \leftarrow \langle \rangle$ → Lista vazia
2. $\alpha \leftarrow d$ → v atual é destino d
3. while $\alpha \neq \text{null}$ do → enquanto $v \neq \text{nulo}$
4. $p \leftarrow \langle \alpha \rangle \cdot p$ → add v atual no início
5. $\alpha \leftarrow A\alpha$ → v atual agora é o antecessor
6. return p → retorna o caminho